From C to C++ I

Objects, references, and namespaces

Roberto Masocco
roberto.masocco@uniroma2.it

Tor Vergata University of Rome
Department of Civil Engineering and Computer Science Engineering

April 23, 2026

Roadmap

1. The C-to-C++ transition
2. Classes
3. RAII
4. Inheritance and virtual dispatch
5. References
6. Namespaces

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

C and C++

What is the relationship?

C++ was designed as a **superset of C**: almost all valid C code is also valid C++ code.

Everything you know from C is still there:

- primitive types, arrays, and **pointers**;
- **structs**, unions, and enumerations;
- operators;
- the **standard C library** (`<cstdio>`, `<cstdlib>`, `<cstring>`...);
- manual memory management with `malloc`/`free`;
- **preprocessor** directives (`#include`, `#define`, `#ifdef`, `#endif`...).

Dust off your C skills. C++ adds **a lot** on top of them.

The C-to-C++ transition

What changes

Some conventions change when moving to C++.

File extensions reflect the language:

- `.cpp` (or `.cc`) for source files;
- `.hpp` (or `.hh`) for header files.

Include guards are replaced by the cleaner pragma:

- `#pragma once` at the top of every header file.

Compilation uses a C++-aware compiler and specifying the **language standard** is relevant:

- `g++ -std=c++17 -o my_program my_program.cpp`

This course targets **C++17**, the standard required by ROS 2 Jazzy.

The C-to-C++ transition

What is added

C++ additions are designed to support **large-scale**, **maintainable** software; we need to cover:

- **classes**: encapsulation, constructors, destructors, member functions;
- **RAII**: the most important C++ programming idiom;
- **inheritance** and **virtual dispatch**: polymorphism with a concrete implementation;
- **references**: a safer, cleaner alternative to pointers in many contexts;
- **namespaces**: organizing and partitioning large codebases;
- **templates**: generic programming, the foundation of the standard library;
- **smart pointers** and the **standard library**: safer memory and modern containers;
- **lambdas** and **callbacks**: the backbone of event-driven programming;
- **concurrency**: `std::thread`, mutexes, condition variables, and atomics.

The C-to-C++ transition

This lecture is here.
Code examples are available in the `tools/cpp-examples/` subfolders of the materials.
Every lecture ends with **examples** showing new language features and an **exercise**.

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

Classes

`class` vs `struct`

C++ keeps the `struct` keyword from C, but gives it new powers: **member functions**, **constructors**, and **access control**.

It also introduces `class`. The **only difference** between the two is the **default access specifier**:

- members of a `struct` are **public** by default;
- members of a `class` are **private** by default.

By convention:

- `struct` is used for **plain data aggregates** with no significant behavior;
- `class` is used when **encapsulation** and **operations** matter.

Classes

```
class MyClass : public ParentClass
{
public:
  MyClass();
  ~MyClass();
  int get_int() { return variable_int_member_; }
  // ...
protected:
  // ...
private:
  int variable_int_member_ = 0;
  double variable_double_member_ = 0.0;
  // ...
};

```

Listing 1: Example of a C++ class.

Classes

Access specifiers

Access specifiers control the **visibility** of class members to the outside world.

There are three:

- `public`: accessible from anywhere;
- `protected`: accessible from the class itself and from **derived classes**;
- `private`: accessible **only** from the class itself, **not even from derived classes**.

A specifier applies to all members declared **below it**, until the next specifier.

Good practice is to expose only what is **necessary**:

- **data members** are almost always `private`, accessible via **getters**/**setters**;
- the **public interface** is kept minimal and stable.

Classes

Member functions and the `this` pointer

Functions declared inside a class are called **member functions** (or *methods*).
They have **direct access to all data members of the class**, regardless of access specifiers.

Inside any (non-static) member function, the keyword `this` is an implicit **pointer to the object on which the function was called**.

Member functions that **do not modify the object** should be declared `const`: the **compiler** then enforces the constraint.

Classes

Constructors and destructors

Every class can define **special member functions** that the compiler calls automatically.

The **constructor** is called when an object is **created**:

- its name is the class name, it has no return type;
- it initializes data members and acquires resources;
- a class may have **multiple constructors** with different parameter lists (**overloading**);
- if none is defined, the compiler generates a **default constructor**.

The **destructor** is called when an object is **destroyed**:

- its name is the class name preceded by ~;
- it **releases resources** and **performs cleanup**;
- there is **exactly one** destructor per class, it takes no arguments.

Guarantee

The destructor is always called, regardless of how the object's scope ends — even in the presence of exceptions.

Classes

Separating declaration from definition

In C++, a class is typically **declared** in a header file (`.hpp`) and **defined** in a source file (`.cpp`).

The **header** contains

- the **class declaration** with all member names and types;
- `inline` **function definitions** (small, frequently called functions).

The **source file** contains

- the full **definitions** of all non-inline member functions;
- member function names are **prefixed** with `ClassName::` (their **namespace**).

Why split?

The header is the **public contract** of the class — it is what other translation units include. Keeping implementation in `.cpp` files reduces compile-time dependencies and **hides internals**.

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

RAII

The resource problem

A **resource** is anything that must be **acquired before use** and **released after use**:

- **memory** allocated on the heap (`malloc`/`free` in C, `new`/`delete` in C++);
- **mutex locks** acquired with `pthread_mutex_lock`;
- **file descriptors** opened with `fopen`/`open`;
- **network sockets**, hardware handles, GPU buffers...

In C, the programmer is responsible for releasing every resource on **every possible exit path** from a function — including error paths.

The classic bug

An early `return`, a `goto`, or an unhandled error condition leaves a **resource unreleased**.
In a long-running system this causes **resource leaks** that accumulate over time.

RAII

Resource Acquisition Is Initialization

Definition of RAII

Resource Acquisition Is Initialization (RAII) is a programming idiom in which a resource is **acquired in the constructor** of an object and **released in the destructor**.

Because the destructor is **guaranteed to run** when an object goes out of scope, resource release becomes **automatic and unconditional**:

- no matter how many `return` statements exist in a function;
- no matter which branch of an `if`/`switch` is taken;
- even if an **exception** is thrown.

RAII is the most important C++ programming idiom.
Every major abstraction in the standard library is built on it.

RAII

RAII in practice

The idiom is not a language feature — it is a **design pattern** that the language makes reliable through its object lifetime rules.

Some examples of objects that use this idiom in C++, from your OS course:

- a **mutex wrapper** that locks on construction and unlocks on destruction;
- a **file handle** class that opens on construction and closes on destruction;
- a **memory buffer** class that allocates on construction and deallocates on destruction.

In C++ the standard library provides **ready-made RAII wrappers** for all common resources:

- `std::unique_ptr`, `std::shared_ptr` for heap memory;
- `std::lock_guard`, `std::unique_lock` for mutexes;
- `std::fstream` for files.

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

Inheritance

Inheritance allows a class (**derived**) to build upon another class (**base**), automatically acquiring all of its members.

The syntax mirrors what you know from *e.g.* Python, with one important addition — an **access specifier** on the base class:

- `class Derived : public Base` — the most common form; the public interface of `Base` remains public in `Derived`;
- `protected` and `private` inheritance exist but are rarely used.

A derived class **extends** the base:

- it may **add** new data members and member functions;
- it may **override** member functions defined in the base.

Its constructor must explicitly call a base constructor.

Inheritance

Virtual dispatch

Consider a pointer (or reference) of type `Base *` pointing to an object of type `Derived`.
Which version of an overridden member function should be called?

- Without `virtual`: the **static type** (*i.e.*, `Base`) determines the call — always the base version, decided at **compile time**;
- With `virtual`: the **dynamic type** (*i.e.*, `Derived`) determines the call — the correct override, decided at **runtime**.

Declaring a member function `virtual` in the base class enables **runtime polymorphism**.
Derived classes that override it should mark it `override` — this lets the compiler catch mistakes.

`override` is not required, but you should always use it for clarity.

Destructors are often `virtual` (why? Think. More on this shortly...)

Inheritance

The vtable

Runtime dispatch has to be implemented somehow. The standard mechanism is the **virtual table** (**vtable**).

Each class that has at least one `virtual` function gets a **vtable**: a static array of **function pointers**, one per virtual function, pointing to the correct override for that class.

Each **object** of such a class carries a hidden pointer (**vptr**) to its class's vtable.

A virtual call then becomes

1. dereference the **vptr** to reach the vtable;
2. index into the vtable to get the **function pointer**;
3. **call** through the pointer.

One extra indirection at runtime — a small, constant cost for full **polymorphism**.

Inheritance

Pure virtual functions and abstract classes

A **pure virtual function** is a virtual function **declared with** `= 0` and **not defined in the base class**.

A class with at least one pure virtual function is **abstract**:

- it **cannot be instantiated** directly;
- it acts as an **interface contract** — derived classes must provide implementations for all pure virtual functions, or they too become abstract.

Abstract classes are the primary mechanism for defining **interfaces** in C++.

Virtual destructors

If a class is meant to be used polymorphically (via base pointers), its destructor **must be virtual**. Otherwise, deleting a `Derived` object through a `Base *` will only call the base destructor, potentially **leaking resources**.

Inheritance

dynamic_cast

Sometimes it is necessary to **recover the derived type** from a base pointer or reference — for example, to access a function that exists only in the derived class.

`dynamic_cast<Derived *>(base_ptr)` performs this at **runtime**:

- if the pointed object is of type `Derived` (or a subclass), it **returns a valid pointer**;
- otherwise, it returns `nullptr` (for pointer casts) or **throws** `std::bad_cast` (for reference casts).

It requires **at least one virtual function** in the hierarchy (the vtable is what makes the check possible).

Frequent use of `dynamic_cast` often signals a **design problem**.

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

References

What is a reference?

A **reference** is an **alias** — an alternative name for an already-existing object.

Declared with & after the type:

1. `int x = 42;`
2. `int & ref = x; // ref` and `x` now name the **same object**

References differ from pointers in three important ways:

- a reference **cannot be null** — it must be bound to an object at declaration;
- a reference **cannot be rebound** — once initialized, it always refers to the same object;
- no explicit dereferencing is needed — syntax is **identical to using the object directly**.

A reference is **not** a new object. No copy is made, no new memory is allocated.

References

References as function parameters

The most common use of references is in **function parameter lists**.

Passing by value in C copies the argument — **expensive** for large objects, impossible for non-copyable ones.

Passing by reference gives the function **direct access** to the caller's object:

- **modifiable reference** (`T &`): the function may read and **write** the argument;
- **constant reference** (`const T &`): the function may only **read** the argument.

`const T &` is the idiomatic way to pass **large objects** that should not be modified:

- avoids the cost of a copy;
- the `const` qualifier makes the intent explicit and is enforced by the compiler.

References

References vs pointers

Both refer to an object without copying it, but the choice between them conveys **intent**.

Prefer a **reference** when:

- the referred object **always exists** for the lifetime of the reference;
- **no rebinding** or pointer arithmetic is needed;
- the code should **read as naturally** as if using the object directly.

Prefer a **pointer** when:

- the object may be **absent** (`nullptr` is a valid state);
- **lifetime** (or ownership) **must be managed** explicitly;
- **pointer arithmetic** or array **indexing** is required.

In modern C++, raw pointers are used **only for non-owning observation**.
Ownership is expressed through **smart pointers** — more on this in the next lecture.

Roadmap

1 The C-to-C++ transition

2 Classes

3 RAII

4 Inheritance and virtual dispatch

5 References

6 Namespaces

Namespaces

The naming collision problem

C has a single **global namespace**: every function, variable, and type defined at file scope lives in the same flat space.

In small programs this is manageable. In large codebases that integrate **multiple libraries** — written by different authors, at different times — it becomes a **problem**:

- **two libraries** may define a function named `init()`, a struct named `Node`, or a constant named `MAX_SIZE`;
- the linker cannot resolve the **collision**;
- C works around this with **name prefixes** by convention: `pthread_create`, `PTHREAD_MUTEX_INITIALIZER`, `sched_setscheduler`...

C++ solves this with **namespaces** — a language-level **partitioning** of the global name space.

Namespaces

Namespaces in C++

A **namespace** is a named scope that groups related declarations:

- defined with the `namespace` keyword;
- may contain functions, classes, variables, type aliases, and other namespaces;
- may be **reopened** across multiple files — all declarations in the same named namespace belong to the same scope.

Names inside a namespace are accessed via the **scope resolution operator** `::`

- `std::vector`, `std::cout`, `std::string` — the C++ standard library lives in `std`;
- `rclcpp::Node`, `rclcpp::Publisher` — the ROS 2 client library lives in `rclcpp`.

Namespaces may be **nested**: `rclcpp::executors::MultiThreadedExecutor` — each `::` steps one level deeper, just like dots in a domain name (*e.g.*, `ing.uniroma2.it`).

Namespaces

`using` declarations and directives

Writing the full namespace prefix every time may become verbose.
C++ provides two mechanisms to shorten names.

A `using` **declaration introduces a single name** into the current scope:

- `using std::vector;` — after this, `vector` alone refers to `std::vector`.

A `using namespace` **directive imports all names** from a namespace:

- `using namespace std;` (`std` convenient to write but **dangerous, never do this**).

Namespaces

Taming long names

Namespaces and templates combined may produce **very long type names**.
C++ provides two ways to create **aliases**:

- `typedef` inherited from C: `typedef std::vector<int> IntVec;`
- `using` alias (preferred in modern C++): `using IntVec = std::vector<int>;`

The `using` form is preferred because it is **more readable**, supports templates, and is consistent with the rest of modern C++ syntax.

You will encounter both in existing code — especially in ROS 2, where aliases like:

- `using SharedPtr = std::shared_ptr<Node>;`

are defined in almost every class to shorten usage at the call site.